Universidade Maurício de Nassau
Curso de Ciência da Computação

BuscaPet

Site de adoção e busca de animais perdidos

Manual Técnico — versão 1.0

Kaian Guthierry da Silva — 01617843
Nivaldo José de Arruda Filho — 01486768
Marlon Porto Torres — 01611478

Recife — 2026

Sumário

1. Visão geral da arquitetura
2. Tecnologias, linguagens e bibliotecas
3. Organização dos módulos e componentes
4. Estrutura do banco de dados
5. Requisitos de ambiente e dependências
6. Instalação e configuração
7. Execução
8. Variáveis de ambiente e serviços
9. APIs e integração
10. O componente de processamento (núcleo OpenCL)
11. Segurança
12. Testes e integração contínua
13. Publicação em produção
14. Manutenção e continuidade
15. Limitações conhecidas

Versão do sistema: **1.0.0-rc.1** (Release Candidate). Para quem vai instalar, executar, publicar ou dar continuidade ao desenvolvimento. O uso do sistema pelas pessoas está no **Manual do Usuário** (repositório `BuscaPet-Docs`, pasta `manuais/`).

1. Visão geral da arquitetura

O BuscaPet tem **três serviços** e um banco de dados:

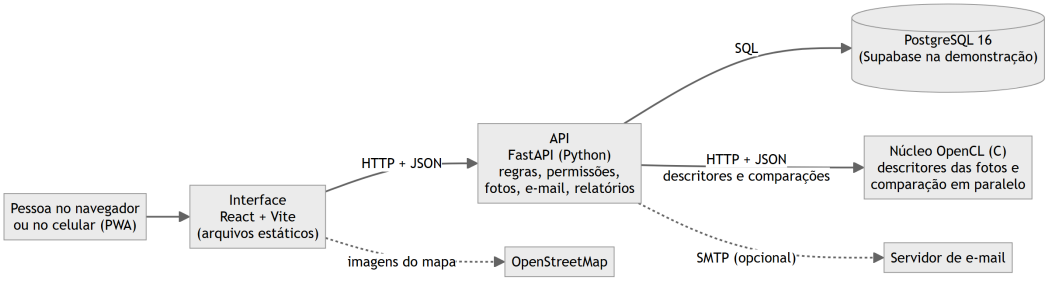


Figura 1 — Arquitetura do BuscaPet: os três serviços e o banco de dados

- **Interface** (`frontend/`): aplicação React de página única, instalável no celular (PWA). Busca todos os dados na API; só as imagens do mapa vêm direto do OpenStreetMap.
- **API** (`api/`): toda regra de negócio, autenticação (JWT), permissões por perfil, armazenamento das fotos, e-mails, relatórios e histórico de operações. É a única que acessa o banco.
- **Núcleo OpenCL** (`nucleo-opencl/`): serviço separado, em C, que transforma cada foto num **descritor** (224 números) e compara descritores em massa, em paralelo nos núcleos da CPU. A API nunca executa OpenCL: chama o núcleo por HTTP. Se o núcleo estiver fora do ar, a API continua funcionando e a busca por foto cai para a ordem por distância.
- **Banco**: PostgreSQL 16. A estrutura é criada pelas migrações do Alembic a partir dos modelos do código.

2. Tecnologias, linguagens e bibliotecas

Camada	Linguagem	Principais bibliotecas
Interface	JavaScript (React 18)	Vite 5 (build), React Router 7, Leaflet (mapas OpenStreetMap), service worker (PWA)
API	Python 3.12	FastAPI 0.158, SQLAlchemy 2, Alembic, Pydantic 2, PyJWT, bcrypt, Pillow 13 (fotos), httpx (cliente do núcleo)
Núcleo	C (C11) + OpenCL C	OpenCL 3.0 via **PoCL** (roda na CPU, sem GPU), libmicrohttpd (HTTP), cJSON
Banco	SQL	PostgreSQL 16
Testes	—	pytest + pytest-cov, Playwright (ponta a ponta), axe-core (acessibilidade), ruff (estilo)
Infraestrutura	—	Docker e Docker Compose, GitHub Actions

As versões exatas estão em `api/requirements.txt`, `frontend/package.json` e `nucleo-opencl/Dockerfile`.

3. Organização dos módulos e componentes

```
BuscaPet-App/
  api/
    app/main.py            cria a aplicação, CORS, registra os roteadores, /health
    app/core/              config (variáveis de ambiente), seguranca (senhas e JWT), deps (quem está logado e
                           permissões por perfil), fotos (validação e gravação), nucleo (cliente HTTP do núcleo),
                           semelhanca (busca e alertas), pontuacao (ranking), auditoria (histórico), email, geo
    app/routers/           uma rota por assunto: auth, usuarios, animais, interesses, abrigos, reencontro, fotos,
                           doacoes, apoiadores, notificacoes, denuncias, admin, relatorios, desempenho (núcleo)
    app/models/            tabelas (SQLAlchemy)
    app/schemas/           formatos de entrada e saída (Pydantic): validação dos dados
    app/scripts/           popular_demo, criar_admin, indexar_fotos, preparar_banco_teste, gerar_doc_api, gerar_mer
    alembic/versions/      migrações 0001 a 0005
    tests/                 testes da API (pytest)
  frontend/
    src/App.jsx            rotas das telas, cabeçalho, rodapé, proteção por perfil
    src/api.js             chamadas à API (token, erros, sem internet)
    src/auth.jsx           sessão (login, logout, conta atual)
    src/pages/             uma tela por arquivo
    src/componentes/       peças reutilizadas (menu, sino, mapa, fotos, doação, meta…)
    src/rotulos.js         textos e formatos (status, dinheiro, datas)
    e2e/                   testes de ponta a ponta (Playwright)
  nucleo-opencl/
    src/servidor.c         serviço HTTP
    src/descritor.c|h      versão sequencial (referência) dos descritores e da comparação
    kernels/               código OpenCL que roda em paralelo (descritor.cl, comparar.cl)
    bench/                 curvas de desempenho medidas
  docs/                    schema.sql, mer.md, api.md, este manual
  .github/workflows/ci.yml integração contínua
  docker-compose.yml       ambiente de desenvolvimento completo
```

Fluxo de uma requisição na API: roteador → dependências (`get_db` abre a sessão do banco; `get_usuario_atual` lê o token; `exigir_tipo(...)` confere o perfil) → validação do corpo pelo schema Pydantic → regra de negócio → `auditoria.registrar` nas ações sensíveis → `commit` → resposta no schema de saída.

4. Estrutura do banco de dados

19 tabelas e 27 chaves estrangeiras, em cinco grupos: contas e perfis (`usuarios`, `tutores`, `abrigos`, `apoiadores`, `membros_equipe_abrigo`), adoção (`animais`, `interesses`, `denuncias`), reencontro (`animais_perdidos`, `avistamentos`, `buscas_salvas`, `correspondencias`, `fotos`, `descritores_visuais`), doações (`necessidades`, `doacoes`, `pontuacoes_apoiador`) e transversais (`notificacoes`, `logs_auditoria`).

- **Diagrama:** `docs/mer.md` (completo e por módulo), gerado dos modelos.
- **SQL:** `docs/schema.sql`, conferido contra os modelos por teste, coluna a coluna.
- **Migrações:** `api/alembic/versions/` (0001 esquema inicial; 0002 alinhamento; 0003 mensagem e unicidade do interesse; 0004 anúncio removido pela moderação; 0005 doação de tutor, valor em dinheiro e meta mensal).

Convenções: chaves estrangeiras nomeadas `fk_<tabela>_<coluna>_<destino>`; datas em UTC no banco (a interface e os relatórios mostram no fuso de Recife); dinheiro em **centavos inteiros**; nada é apagado quando há histórico (conta fica `ativo = false`, anúncio ganha `removido_em`).

5. Requisitos de ambiente e dependências

Para	Precisa
API	Python **3.12** e um PostgreSQL 16 (local, Docker ou Supabase)
Interface	Node.js **20 ou mais novo**
Núcleo	**Docker** (o compilador, o PoCL e as bibliotecas ficam dentro da imagem)
Testes ponta a ponta	PostgreSQL local (cria o banco `buscapet_e2e` sozinho) e o navegador do Playwright (`npx playwright install chromium`)

Memória: com tudo rodando (Docker, API, interface e testes) uma máquina de 8 GB fica no limite; rode o ponta a ponta com `--workers=2` e desligue a API e a interface de desenvolvimento antes.

6. Instalação e configuração

```
git clone https://github.com/Equipe-BuscaPet/BuscaPet-App.git
cd BuscaPet-App

# API
cd api
python -m venv .venv
.venv\Scripts\activate            # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
# crie api/.env (veja a seção 8), depois:
set DATABASE_URL=...              # bash: export DATABASE_URL=...  (o Alembic lê a variável de ambiente)
alembic upgrade head              # cria ou atualiza as tabelas
python -m app.scripts.popular_demo   # dados de demonstração (só em banco local; senha das contas: Senha1234!)
python -m app.scripts.criar_admin --nome "Seu Nome" --email voce@exemplo.com

# Interface
cd ../frontend
npm install

# Núcleo
cd ..
docker compose up -d --build nucleo-opencl
```

Alternativa com tudo em containers: crie o `.env` na raiz (seção 8) e rode `docker compose up -d --build db api frontend nucleo-opencl`.

7. Execução

```
# API (pasta api/, ambiente virtual ativo)
uvicorn app.main:app --port 8000        # http://localhost:8000/docs  ·  /health/db

# Interface (pasta frontend/)
npm run dev                             # http://localhost:5173

# Núcleo
docker compose up -d nucleo-opencl      # http://localhost:8001/saude
```

Com o núcleo fora do Docker Compose, defina `NUCLEO_OPENCL_URL=http://127.0.0.1:8001` no `api/.env`. Fotos enviadas com o núcleo desligado ficam sem descritor: depois, `python -m app.scripts.indexar_fotos` (ou o botão em Administração → Núcleo OpenCL).

8. Variáveis de ambiente e serviços

Nenhuma credencial fica no repositório: os arquivos `.env` estão no `.gitignore` e fora das imagens Docker (`.dockerignore`). Valores reais ficam só no ambiente de cada máquina e no painel da hospedagem.

API (`api/.env` ou variáveis do serviço):

Variável	Obrigatória	Para quê	Exemplo
`DATABASE_URL`	sim	conexão com o PostgreSQL	`postgresql+psycopg://USUARIO:SENHA@HOST:5432/BANCO`
`JWT_SECRET`	**sim fora do banco local**	assina os logins. Com o valor de exemplo, a API **se recusa a subir** num banco que não seja local	gerar com `python -c "import secrets; print(secrets.token_urlsafe(48))"`
`NUCLEO_OPENCL_URL`	não	endereço do núcleo	`http://nucleo-opencl:8001` (padrão)
`CORS_ORIGENS`	em produção	endereços da interface que podem chamar a API, em **formato JSON**	`["https://buscapet.exemplo.app"]`
`URL_INTERFACE`	em produção	endereço da interface, usado nos links dos e-mails	`https://buscapet.exemplo.app`
`SMTP_HOST`, `SMTP_PORTA`, `SMTP_USUARIO`, `SMTP_SENHA`, `EMAIL_REMETENTE`	não	envio real de e-mail (recuperação de senha, avisos). Sem `SMTP_HOST`, os e-mails viram arquivos em `api/caixa-de-saida/`	—
`PASTA_FOTOS`	não	onde as fotos são gravadas (precisa ser um disco **persistente** em produção)	`/dados/fotos`

Variável	Obrigatória	Para quê	Exemplo
`PORT`	não	porta da API na imagem Docker (as hospedagens costumam definir sozinhas)	`8000`

Interface (no momento do build): `VITE_API_URL` = endereço público da API (padrão `http://localhost:8000`).

Núcleo: `PORTA` (padrão 8001), `NUCLEO_KERNELS` (pasta dos kernels; já definida na imagem).

9. APIs e integração

- **API do BuscaPet** (própria): 77 rotas, todas descritas em `docs/api.md` (gerado do código: quem pode usar, parâmetros, corpo, respostas e erros de cada uma) e no Swagger (`/docs`). Autenticação: `POST /auth/login` (formulário `username` = e-mail, `password`) devolve um token JWT de 24 horas, enviado depois no cabeçalho `Authorization: Bearer <token>`. Perfis: tutor, abrigo, apoiador e admin; `401` sem login, `403` sem permissão.
- **Núcleo OpenCL** (próprio): `GET /saude`, `POST /extrair-descritor`, `POST /comparar`, `GET /benchmark`; contrato em `nucleo-opencl/README.md`. Cliente na API: `api/app/core/nucleo.py`, com tempo limite e um **disjuntor**: depois de uma falha, a API passa 30 segundos sem tentar o núcleo, para as telas não ficarem esperando.
- **Serviços externos** (só infraestrutura, sem API de dados de terceiros): PostgreSQL (Supabase), imagens do mapa do OpenStreetMap (baixadas pelo navegador), servidor SMTP (opcional). O botão de WhatsApp é só um link `wa.me`.

10. O componente de processamento (núcleo OpenCL)

É a parte de **processamento paralelo** do projeto (disciplina de Tópicos Avançados), usada no reencontro de animais perdidos (RF-21 a RF-29).

- **Descritor (RF-21):** cada foto é reduzida a 128×128, tem a iluminação corrigida e o animal separado do fundo, e vira 224 números: histograma de **cor** (52 tons × 4 regiões), **textura** da pelagem (LBP, 10 classes) e **proporções** do animal. Toda a conta é feita com inteiros, por isso a versão paralela devolve exatamente os mesmos números que a sequencial.
- **Comparação (RF-22, RF-25):** qui-quadrado entre os histogramas e diferença de proporções → nota de 0 a 1 → semelhança **alta** (≥ 0,75), **média** (≥ 0,55) ou **baixa**. O sistema nunca afirma que encontrou o animal: a nota só ordena os candidatos; quem reconhece é a pessoa.
- **Onde é usado:** busca a partir de um perdido ou de um avistamento (`GET /reencontro/semelhantes`); **monitoramento contínuo** (cada foto nova é comparada com as buscas salvas e gera alerta ao tutor); alerta ao abrigo quando um animal parecido com um perdido chega. O monitoramento acontece no momento do registro, dentro da própria requisição (não há fila separada).
- **Paralelismo:** na extração, cada item de trabalho do OpenCL processa um bloco de 8 linhas da imagem; na comparação, cada item compara um par (foto buscada × candidato), e milhares de pares rodam ao mesmo tempo. Os kernels leem os dados sem cópia (`CL_MEM_USE_HOST_PTR`), já que o dispositivo é a própria CPU (PoCL).
- **Desempenho medido (RF-29):** i5-2400, 4 núcleos: busca 1 × 100.000 em 3,3×; lote 100 × 10.000 em 3,8×; extração de 1.000 fotos em 2,1× (kernel 3,1×). Com pouco volume o OpenCL perde (custo fixo); com volume, o ganho se aproxima do número de núcleos (Lei de Amdahl). A comparação fica visível em Administração → Núcleo OpenCL (`GET /admin/nucleo/benchmark`), que também confere que os dois modos dão o mesmo resultado.
- **Testes do núcleo:** `make teste` (dentro do build da imagem) confere a equivalência sequencial × OpenCL; a imagem só é gerada se passar. A CI sobe o núcleo e roda a API contra ele com fotos reais.

11. Segurança

- **Senhas** com bcrypt; login com tempo constante (não revela quais e-mails existem); recuperação de senha por link de uso único com validade de 30 minutos.
- **Tokens JWT** (PyJWT, HS256) com validade de 24 horas; o token de redefinição de senha não serve como login.
- **Permissões por perfil** em cada rota (`exigir_tipo`), conferidas por uma matriz de testes (rotas × perfis).
- **Fotos:** toda foto é aberta e regravada pelo servidor (confirma o formato, limita o tamanho, remove os metadados, inclusive a **localização GPS** do celular); nome aleatório.
- **Localização aproximada:** perdidos e avistamentos mostram ao público um ponto aproximado; o local exato só para quem registrou.
- **Redirecionamento seguro:** o retorno depois do login só aceita caminhos internos (`destinoInterno` em `frontend/src/auth.jsx`).
- **Segredos:** fora do Git e das imagens; a API recusa subir com o `JWT_SECRET` de exemplo num banco remoto.
- **Dependências:** auditadas na Sprint 11 (`pip-audit`, `npm audit`); veja a seção 15 sobre o que restou.

12. Testes e integração contínua

Comando	O que verifica
`cd api && pytest --cov=app`	regras de negócio, permissões, validações, documentação em dia (guia da API, MER, schema.sql); cobertura mínima de 90%
`cd api && ruff check .`	estilo e erros comuns
`cd frontend && npm run build`	a interface compila
`cd frontend && npm run e2e`	navegador de verdade contra a API e um PostgreSQL próprio (`buscapet_e2e`), no computador e no celular, com varredura de todas as telas e acessibilidade (axe-core)
`docker compose build nucleo-opencl`	compila e testa o núcleo (equivalência sequencial × OpenCL)

A CI (`.github/workflows/ci.yml`) roda tudo a cada Pull Request, em quatro trabalhos: `api`, `frontend`, `e2e` e `nucleo-opencl`.

13. Publicação em produção

A publicação precisa de quatro peças. O que vale para qualquer hospedagem:

Peça	Como publicar	Atenção
Banco	Supabase (PostgreSQL)	rodar `alembic upgrade head` com a `DATABASE_URL` de produção antes de subir a API; o plano gratuito **pausa** após cerca de uma semana sem uso (acessar o painel e restaurar)
API	imagem `api/Dockerfile` (já roda sem `--reload`, como usuário sem privilégios, na porta `$PORT`)	**as fotos ficam em disco** (`PASTA_FOTOS`): a hospedagem precisa de **volume persistente**, senão as fotos somem a cada nova publicação; definir `JWT_SECRET`, `CORS_ORIGENS`, `URL_INTERFACE` e `NUCLEO_OPENCL_URL`
Núcleo	imagem `nucleo-opencl/Dockerfile` (cerca de 850 MB)	precisa de memória para o PoCL (reserve pelo menos 1 GB); se ficar em rede interna da hospedagem, a API o alcança pelo nome do serviço
Interface	`npm run build` com `VITE_API_URL` = endereço da API → publicar a pasta `dist/` em hospedagem estática	as regras para recarregar páginas internas já estão no projeto: `frontend/vercel.json` (Vercel) e `public/_redirects` (Netlify, Render)

Depois da primeira publicação: `python -m app.scripts.criar_admin` (administrador) e `python -m app.scripts.indexar_fotos` (descritores das fotos). Os dados de demonstração (`popular_demo`) foram feitos para banco local e se recusam a rodar num banco remoto, porque as senhas deles são públicas. Para a apresentação, o caminho recomendado é cadastrar abrigos, animais e contas pela própria interface; se a equipe decidir usar o cenário de demonstração em produção, `--forcar` existe, e as senhas dessas contas precisam ser trocadas logo depois. Conferir `/health/db` na API, `/saude` no núcleo e a aba Administração → Núcleo OpenCL ("No ar").

14. Manutenção e continuidade

- **Mudou um modelo (tabela/coluna)?** `alembic revision --autogenerate -m "o que mudou"`, revisar o arquivo gerado, `alembic upgrade head`, atualizar `docs/schema.sql` e rodar `python -m app.scripts.gerar_mer`. O teste `test_doc_banco.py` acusa se faltar algum passo.
- **Mudou ou criou uma rota?** Escrever a docstring da função (vira a descrição no Swagger e no guia) e rodar `python -m app.scripts.gerar_doc_api`. O teste `test_doc_api.py` acusa se faltar.
- **Nova permissão:** usar `exigir_tipo(TipoConta....)` na rota e acrescentar a linha na matriz `tests/test_permissoes.py`.
- **Ações sensíveis:** registrar com `auditoria.registrar(...)` (aparecem no histórico de operações).
- **Datas:** gravar em UTC; mostrar no fuso de Recife. Consultas novas: testar também no PostgreSQL (o SQLite dos testes aceita coisas que o PostgreSQL recusa).
- **Fluxo de trabalho:** branch por tarefa, commits pequenos em português, Pull Request com a CI verde antes de integrar; versões com tag (`v1.0.0`).

15. Limitações conhecidas

Limitação	Motivo	Impacto
Pagamento não acontece dentro da plataforma; doação em dinheiro é por PIX direto ao abrigo, confirmada por ele	fora do escopo (exige intermediador de pagamento)	o abrigo confirma manualmente cada PIX
Tabela `membros_equipe_abrigo`, coluna `abrigos.documento_comprobatorio_url` e o tipo de correspondência `busca_inversa` existem no modelo, mas não têm tela	previstos na modelagem da Sprint 2; ficaram para depois da versão 1.0	nenhum: não interferem no uso
Fotos gravadas no disco do servidor da API	simplicidade; a troca por um armazenamento externo fica isolada em `app/core/fotos.py`	a hospedagem precisa de disco persistente (seção 13)
Alerta do `esbuild` (ferramenta do servidor de desenvolvimento do Vite)	a correção exige trocar o Vite 5 pelo 8	só afeta `npm run dev`; o site publicado (`npm run build`) não usa esse servidor
Núcleo testado em CPU (PoCL), não em GPU	a hospedagem e as máquinas da equipe não têm GPU dedicada	o código é OpenCL padrão e roda em GPU sem mudanças, se houver